

Network Traffic Analysis

Wireshark practice lab

Goal

I used this capture to practice reading common network traffic in Wireshark instead of only learning the protocols from notes. The capture contains 15 packets.

Device	Address
Client	192.168.56.10
Gateway	192.168.56.1
DNS server	192.168.56.53
Web server	203.0.113.80

Packet analysis

ARP - packets 1 and 2

The client first asks for the MAC address of 192.168.56.1. Packet 1 is the ARP request and packet 2 is the reply from the gateway. This helped me connect the idea that the client needs a Layer 2 address for the next hop before it can send the frame on the local network.

DNS - packets 3 and 4

The client sends a DNS query to 192.168.56.53 asking for the A record of training.example. The reply returns 203.0.113.80. The DNS request uses UDP destination port 53 and the client source port is 53000.

TCP handshake - packets 5 to 7

The client connects from port 51514 to server port 80. The handshake is SYN, SYN/ACK, then ACK. I used this part to practice recognizing TCP flags and following the start of a connection in Wireshark.

HTTP - packets 8 and 9

Packet 8 contains a GET request for /lab/index.html and packet 9 contains an HTTP/1.1 200 OK response. The main security point I took from this is that the request is readable directly because the traffic is not encrypted.

TCP close - packets 10 to 13

After the response, the connection is closed using FIN/ACK packets. Looking at the teardown made it clearer that TCP closes the connection in both directions rather than simply disappearing after the response.

ICMP - packets 14 and 15

The final two packets are an ICMP echo request and echo reply. This is the basic request/reply behavior used by ping. A reply confirms reachability for these packets, not that every service on the destination is available.

Filters used

```
arp
dns
tcp.port == 80
http
icmp

tcp.flags.syn == 1 && tcp.flags.ack == 0
```

What I learned

The most useful part of this lab was seeing how the protocols fit together instead of treating them as separate topics. The DNS response gives the client the server IP, TCP establishes the connection, and HTTP is then carried inside that TCP connection. I also got more comfortable checking packet numbers, ports, TCP flags, and following a conversation with display filters.

Limits of this lab

This is a small practice capture. It does not contain malicious traffic, TLS/HTTPS, IPv6, retransmissions, or a large number of hosts. I would need a more realistic capture before making security conclusions.